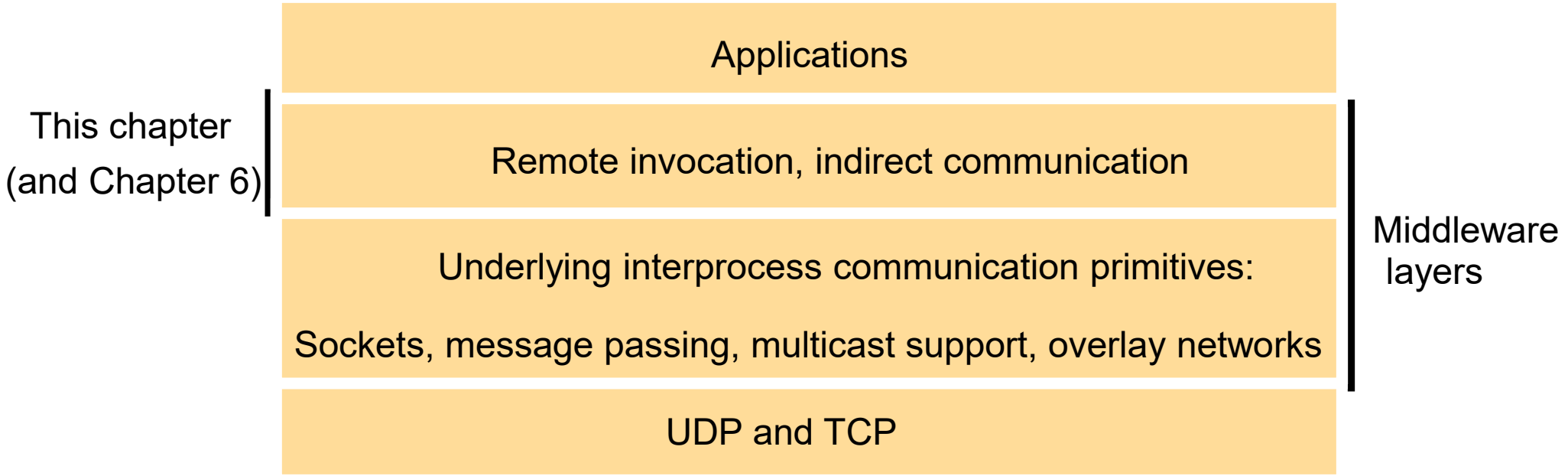
PARALLEL AND DISTRIBUTED COMPUTING

CHAPTER 5: REMOTE INVOCATION

LESSON TOPICS

- Introduction
- Request-reply protocols
- Remote procedure call
- Remote method invocation
- Case study: Java RMI

MIDDLEWARE LAYERS



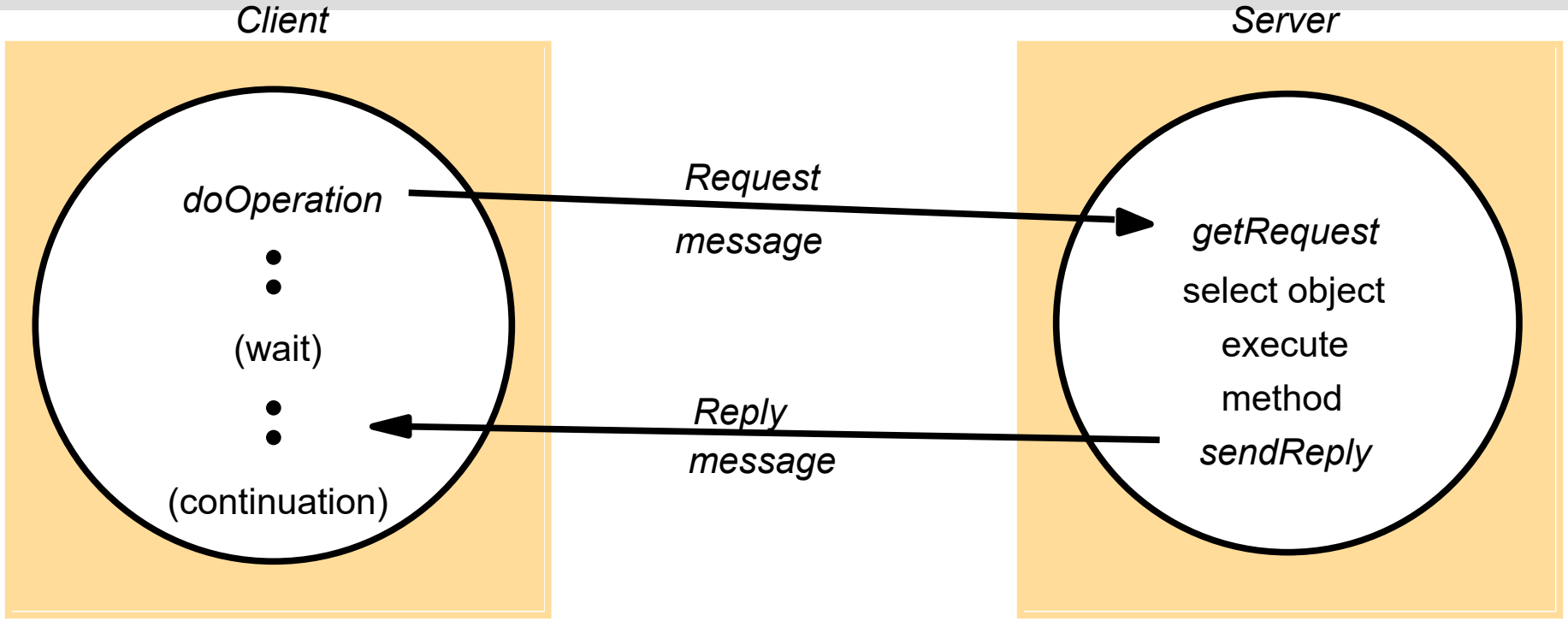
REQUEST-REPLY PROTOCOL

- Request-reply communication is synchronous because the client process blocks until the reply arrives from the server.
- It can also be reliable because the reply from the server is effectively an acknowledgement to the client.
- Asynchronous request-reply communication is an alternative that may be useful in situations where clients can afford to retrieve replies later

Request-reply protocol

- The protocol is based on a trio of communication primitives, doOperation, getRequest and sendReply.
- This request-reply protocol matches requests to replies. It may be designed to provide certain delivery guarantees.
- If UDP datagrams are used, the delivery guarantees must be provided by the request-reply protocol, which may use the server reply message as an acknowledgement of the client request message.

REQUEST-REPLY COMMUNICATION



OPERATIONS OF THE REQUEST-REPLY PROTOCOL

public byte[] doOperation (RemoteRef s, int operationId, byte[] arguments)

sends a request message to the remote server and returns the reply.
The arguments specify the remote server, the operation to be invoked and the arguments of that operation.

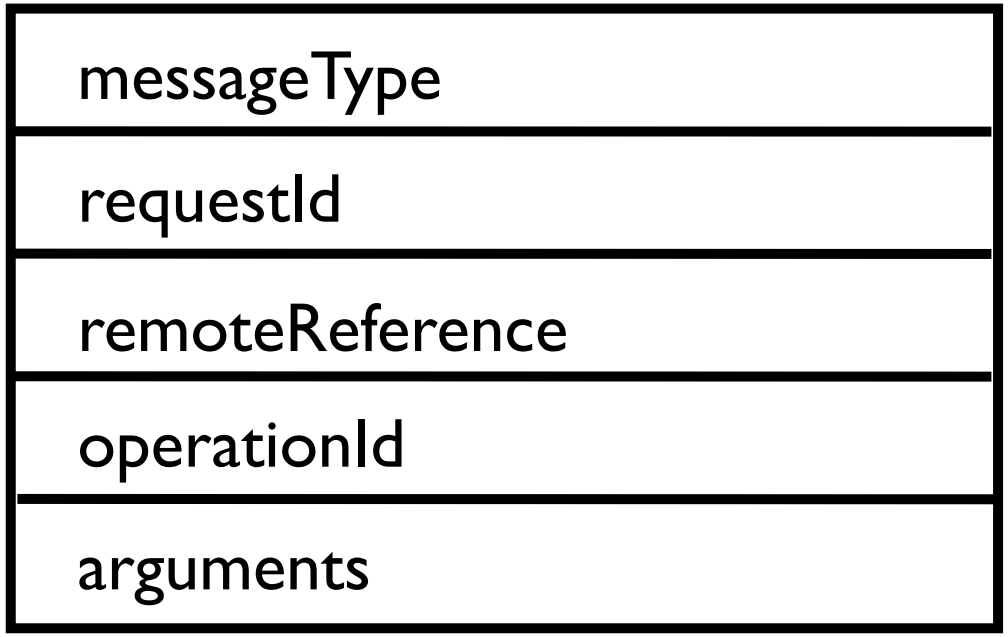
public byte[] getRequest ();

acquires a client request via the server port.

public void sendReply (byte[] reply, InetAddress clientHost, int clientPort);

sends the reply message reply to the client at its Internet address and port.

REQUEST-REPLY MESSAGE STRUCTURE



RPC EXCHANGE PROTOCOLS

- the request (R) protocol

  In the R protocol, a single Request message is sent by the client to the server. The R protocol may be used when there is no value to be returned from the remote operation and the client requires no confirmation that the operation has been executed. The client may proceed immediately after the request message is sent as there is no need to wait for a reply message. This protocol is implemented over UDP datagrams

- the request-reply (RR) protocol;

  The RR protocol is useful for most client-server exchanges because it is based on the request-reply protocol. Special acknowledgement messages are not required, because a server's reply message is regarded as an acknowledgement of the client's request message

- the request-reply-acknowledge reply (RRA) protocol

  The RRA protocol is based on the exchange of three messages: request-reply acknowledge reply. The Acknowledge reply message contains the requestId from the reply message being acknowledged. This will enable the server to discard entries from its history

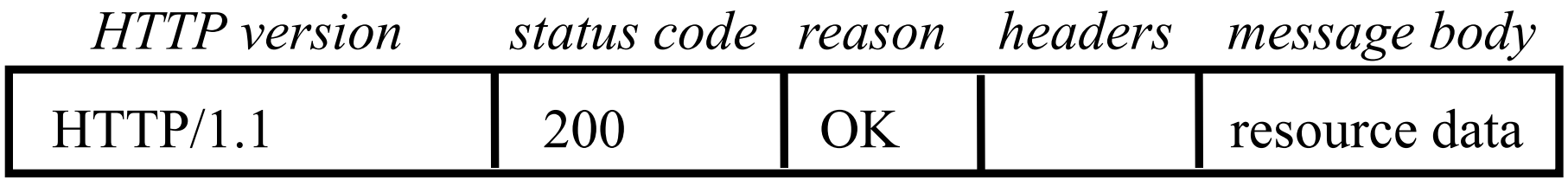
RPC EXCHANGE PROTOCOLS

Name	*Messages sent by*		
	Client	*Server*	*Client*
R	*Request*		
RR	*Request*	*Reply*	
RRA	*Request*	*Reply*	*Acknowledge reply*

HTTP REQUEST MESSAGE

method	*URL or pathname*	*HTTP version*	*headers*	*message body*
GET	//www.dcs.qmw.ac.uk/index.html	HTTP/ 1.1		

HTTP REPLY MESSAGE



INTERFACES IN DISTRIBUTED SYSTEMS

- In a distributed program, the modules can run in separate processes. In the client-server model, in particular, each server provides a set of procedures that are available for use by clients.
- The term service interface is used to refer to the specification of the procedures offered by a server, defining the types of the arguments of each of the procedures.
- Interface definition languages (IDLs) are designed to allow procedures implemented in different languages to invoke one another. An IDL provides a notation for defining interfaces in which each of the parameters of an operation may be described as for input or output in addition to having its type specified.

CORBA IDL EXAMPLE

```
// In file Person.idl
struct Person {
        string name;
        string place;
        long year;
} ;
interface PersonList {
        readonly attribute string listname;
        void addPerson(in Person p) ;
        void getPerson(in string name, out Person p);
        long number();
};
```

CALL SEMANTICS

Fault tolerance measures			*Call semantics*
Retransmit request message	*Duplicate filtering*	*Re-execute procedure or retransmit reply*	
No	Not applicable	Not applicable	*Maybe*
Yes	No	Re-execute procedure	*At-least-once*
Yes	Yes	Retransmit reply	*At-most-once*

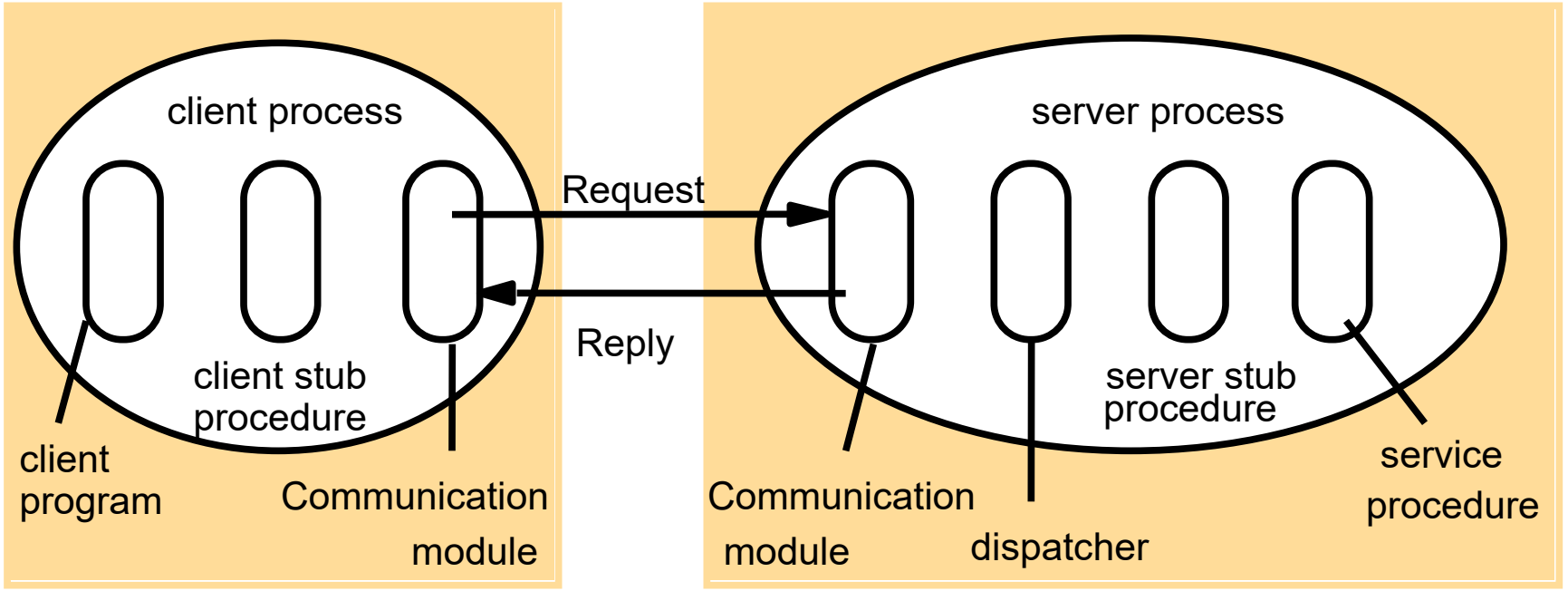
RPC CALL SEMANTICS

- Request-reply protocols were discussed, where we showed that doOperation can be implemented in different ways to provide different delivery guarantees.

main choices are:

- Retry request message: Controls whether to retransmit the request message until either a reply is received or the server is assumed to have failed.
- Duplicate filtering: Controls when retransmissions are used and whether to filter out duplicate requests at the server.
- Retransmission of results: Controls whether to keep a history of result messages to enable lost results to be retransmitted without re-executing the operations at the server.

ROLE OF CLIENT AND SERVER STUB PROCEDURES IN RPC



FILES INTERFACE IN SUN XDR

```
const MAX = 1000;
typedef int FileIdentifier;
typedef int FilePointer;
typedef int Length;
struct Data {
    int length;
    char buffer[MAX];
};
struct writeargs {
    FileIdentifier f;
    FilePointer position;
    Data data;
};
```

```
struct readargs {
    FileIdentifier f;
    FilePointer position;
    Length length;
};

program FILEREADWRITE {
  version VERSION {
    void WRITE(writeargs)=1; 1
    Data READ(readargs)=2;  2
  }=2;
} = 9999;
```

REMOTE METHOD INVOCATION (RMI)

- Remote method invocation (RMI) is closely related to RPC but extended into the world of distributed objects. In RMI, a calling object can invoke a method in a potentially remote object. As with RPC, the underlying details are generally hidden from the user.
- **Object references**: Objects can be accessed via object references. For example, in Java, a variable that appears to hold an object actually holds a reference to that object.
- **Interfaces:** An interface provides a definition of the signatures of a set of methods (that is, the types of their arguments, return values and exceptions) without specifying their implementation. An object will provide a particular interface if its class contains code that implements the methods of that interface.
- **Actions :** Action in an object-oriented program is initiated by an object invoking a method in another object.

REMOTE METHOD INVOCATION (RMI)

- **Exceptions**: Programs can encounter many sorts of errors and unexpected conditions of varying seriousness.
- **Garbage collection**: It is necessary to provide a means of freeing the space occupied by objects when they are no longer needed. A language such as Java, that can detect automatically when an object is no longer accessible recovers the space and makes it available for allocation to other objects. This process is called garbage collection.

DISTRIBUTED OBJECT

- Distributed object systems may adopt the client-server architecture.
- In this case, objects are managed by servers and their clients invoke their methods using remote method invocation.
- In RMI, the client's request to invoke a method of an object is sent in a message to the server managing the object.
- The invocation is carried out by executing a method of the object at the server and the result is returned to the client in another message.
- **Remote object references**: Other objects can invoke the methods of a remote object if they have access to its remote object reference. For example, a remote object reference for B in Figure must be available to A.
- **Remote interfaces**: Every remote object has a remote interface that specifies which of its methods can be invoked remotely.

REMOTE OBJECT REFERENCES

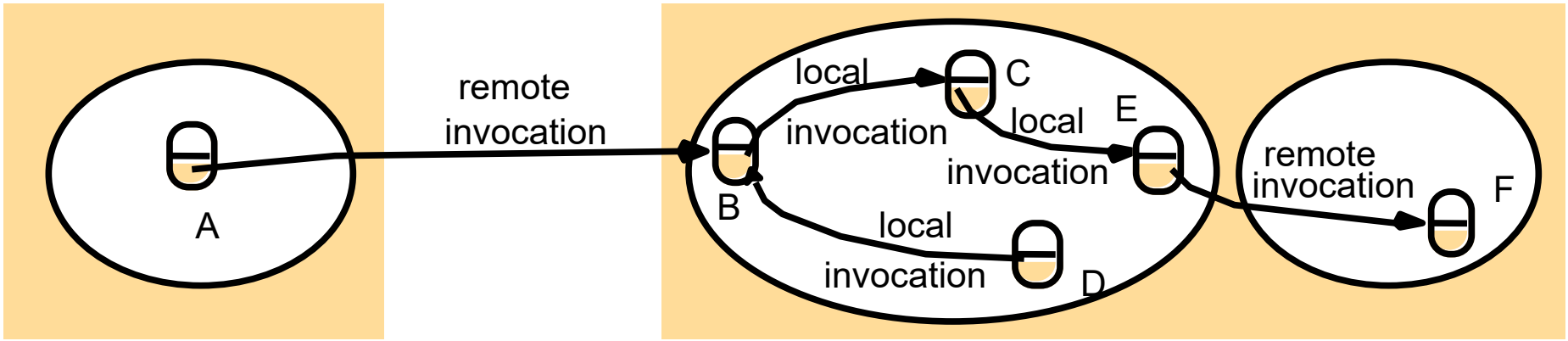
Remote object references:

- The notion of object reference is extended to allow any object that can receive an RMI to have a remote object reference. A remote object reference is an identifier that can be used throughout a distributed system to refer to a particular unique remote object.

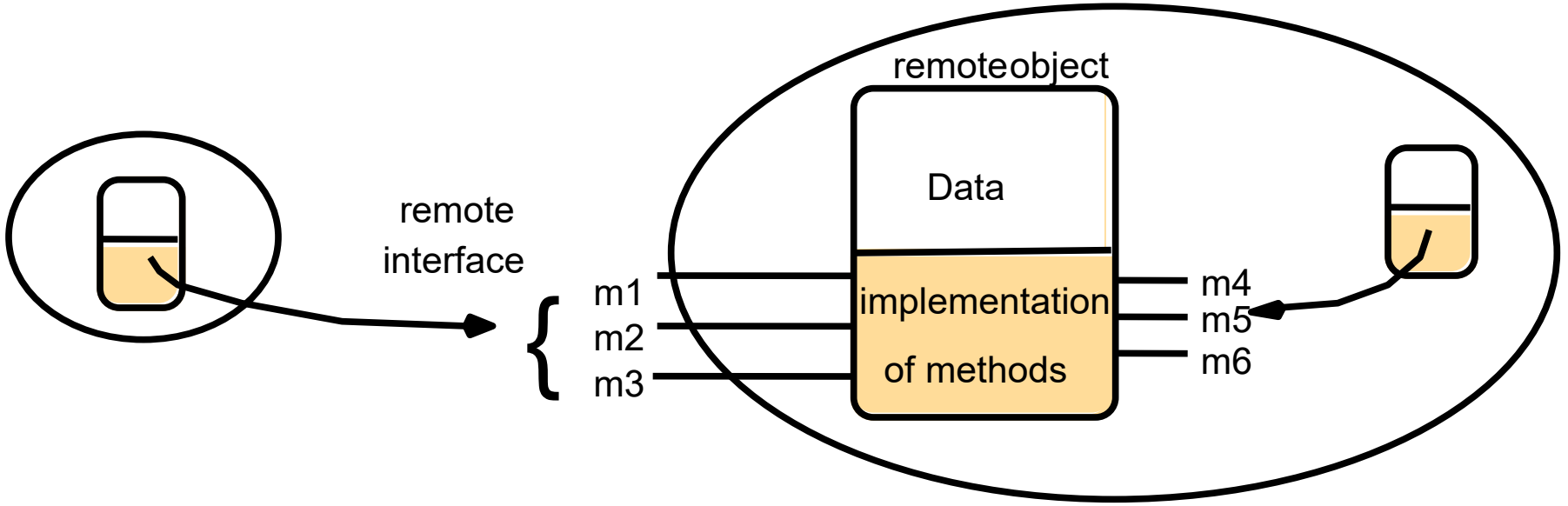
Remote interfaces:

- The class of a remote object implements the methods of its remote interface, for example as public instance methods in Java. Objects in other processes can invoke only the methods that belong to its remote interface.

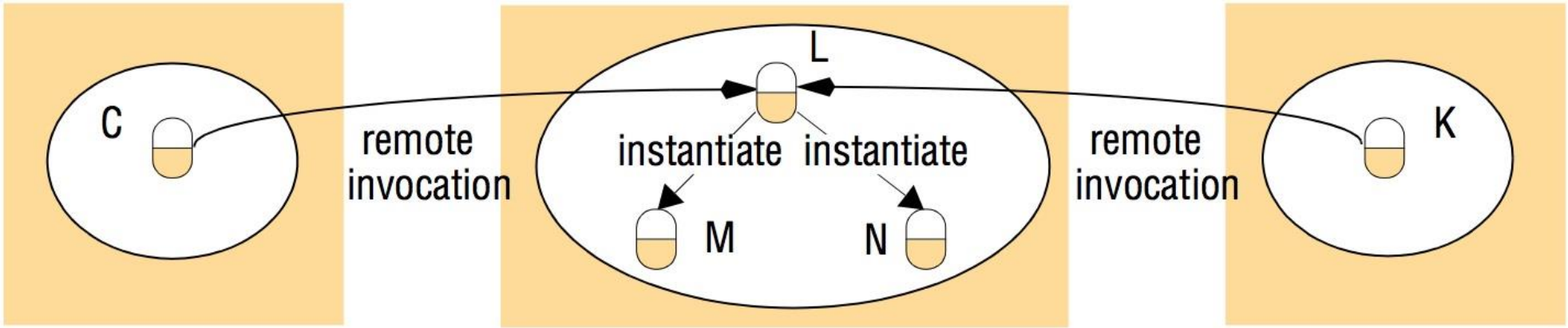
REMOTE AND LOCAL METHOD INVOCATIONS



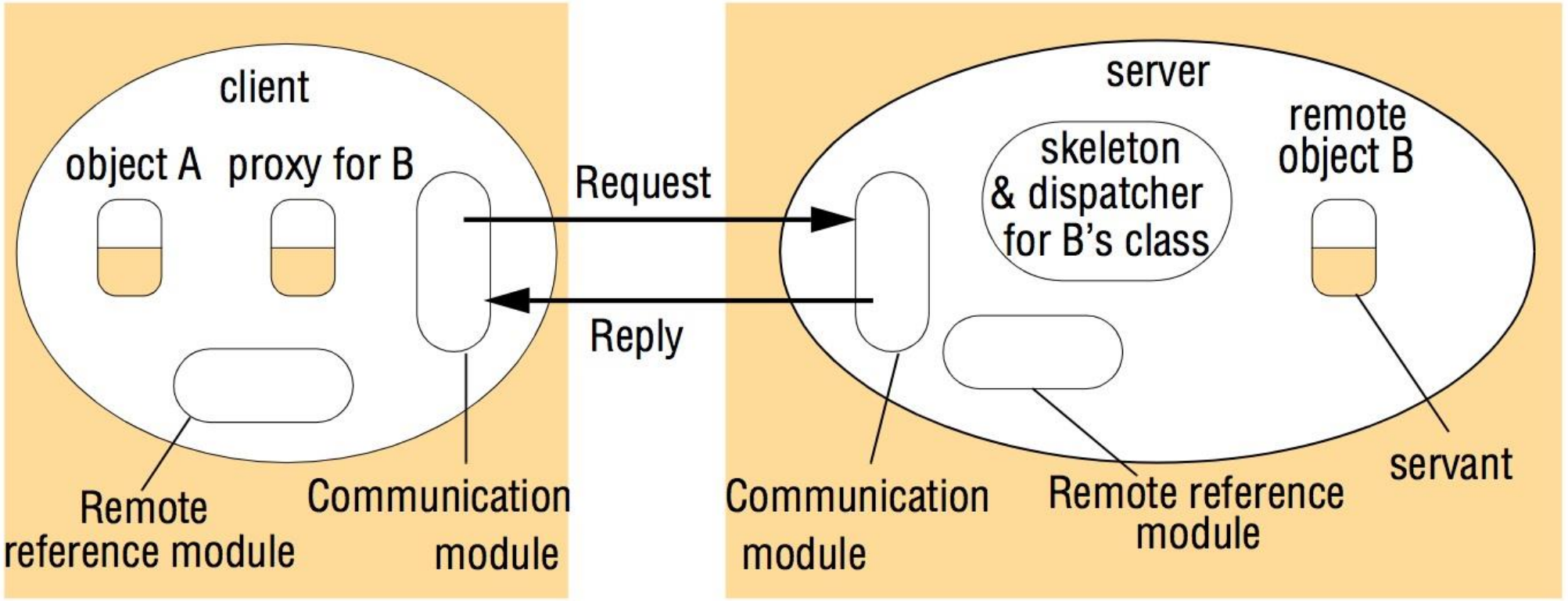
A REMOTE OBJECT AND ITS REMOTE INTERFACE



INSTANTIATION OF REMOTE OBJECTS



THE ROLE OF PROXY AND SKELETON IN REMOTE METHOD INVOCATION



IMPLEMENTATION OF RMI

- Servants • A servant is an instance of a class that provides the body of a remote object. It is the servant that eventually handles the remote requests passed on by the corresponding skeleton. Servants live within a server process. They are created when remote objects are instantiated and remain in use until they are no longer needed, finally being garbage collected or deleted.
- The RMI software
  - Proxy: The role of a proxy is to make remote method invocation transparent to clients by behaving like a local object to the invoker; but instead of executing an invocation, it forwards it in a message to a remote object. It hides the details of the remote object reference, the marshalling of arguments, unmarshalling of results and sending and receiving of messages from the client. There is one proxy for each remote object for which a process holds a remote object reference.

IMPLEMENTATION OF RMI

- RMI software
  - Proxy: The role of a proxy is to make remote method invocation transparent to clients by behaving like a local object to the invoker; but instead of executing an invocation, it forwards it in a message to a remote object. It hides the details of the remote object reference, the marshalling of arguments, unmarshalling of results and sending and receiving of messages from the client. There is one proxy for each remote object for which a process holds a remote object reference.
  - Dispatcher: A server has one dispatcher and one skeleton for each class representing a remote object. In our example, the server has a dispatcher and a skeleton for the class of remote object B.
  - Skeleton: The class of a remote object has a skeleton, which implements the methods in the remote interface. They are implemented quite differently from the methods in the servant that incarnates a remote object.

JAVA REMOTE INTERFACES *SHAPE* AND *SHAPELIST*

```
import java.rmi.*;
import java.util.Vector;
public interface Shape extends Remote {
    int getVersion() throws RemoteException;
    GraphicalObject  getAllState() throws RemoteException;      1
}
public interface ShapeList extends Remote {
    Shape newShape(GraphicalObject g) throws RemoteException; 2
    Vector allShapes() throws RemoteException;
    int getVersion() throws RemoteException;
}
```

THE *NAMING* CLASS OF JAVA RMIREGISTRY

void rebind (String name, Remote obj)

This method is used by a server to register the identifier of a remote object by name, as shown in Figure 15.18, line 3.

void bind (String name, Remote obj)

This method can alternatively be used by a server to register a remote object by name, but if the name is already bound to a remote object reference an exception is thrown.

void unbind (String name, Remote obj)

This method removes a binding.

Remote lookup(String name)

This method is used by clients to look up a remote object by name, as shown in Figure 5.20 line 1. A remote object reference is returned.

String [] list()

This method returns an array of Strings containing the names bound in the registry.

JAVA CLASS *SHAPELISTSERVER* WITH *MAIN* METHOD

```
import java.rmi.*;
public class ShapeListServer{
    public static void main(String args[]){
        System.setSecurityManager(new RMISecurityManager());
        try{
            ShapeList aShapeList = new ShapeListServant();          1
                Naming.rebind("Shape List", aShapeList );           2
            System.out.println("ShapeList server ready");
            }catch(Exception e) {
            System.out.println("ShapeList server main " + e.getMessage());}
    }
}
```

JAVA CLASS *SHAPELISTSERVANT* IMPLEMENTS INTERFACE *SHAPELIST*

```
import java.rmi.*;
import java.rmi.server.UnicastRemoteObject;
import java.util.Vector;
public class ShapeListServant extends UnicastRemoteObject implements ShapeList {
     private Vector theList;  // contains the list of Shapes
     private int version;
    public ShapeListServant()throws RemoteException{...}
    public Shape newShape(GraphicalObject g) throws RemoteException {  1
        version++;
            Shape s = new ShapeServant( g, version);                2
            theList.addElement(s);
            return s;
    }
    public  Vector allShapes()throws RemoteException{...}
    public int getVersion() throws RemoteException { ... }
}
```

JAVA CLIENT OF *SHAPELIST*

```
import java.rmi.*;
import java.rmi.server.*;
import java.util.Vector;
public class ShapeListClient{
    public static void main(String args[]){
      System.setSecurityManager(new RMISecurityManager());
      ShapeList aShapeList = null;
      try{
            aShapeList  = (ShapeList) Naming.lookup("//bruno.ShapeList") ;      1
            Vector sList = aShapeList.allShapes();                    2
      } catch(RemoteException e) {System.out.println(e.getMessage());
      }catch(Exception e) {System.out.println("Client: " + e.getMessage());}
    }
}
```

CLASSES SUPPORTING JAVA RMI